UNIVERSITATEA TEHNICĂ DIN CLUJ-NAPOCA

FACULTATEA DE AUTOMATICĂ ȘI CALCULATOARE

SPECIALIZAREA: CALCULATOARE ȘI TEHNOLOGIA INFORMAȚIEI

Structura Sistemelor de Calcul

MultiCore Benchmark Studio

Evaluarea comparativă a performanței unui procesor multicore în C și Rust pe arhitectura x86-64

Student: Florea Adelin-Mihai
Grupa: 30233
Profesor laborant: Neagu Mădălin

Cluj-Napoca, 2025–2026

Cuprins

1 Introducere **3**
1.1 Contextul actual al arhitecturilor multicore . . . 3
1.2 Propunerea de proiect . . . 3
1.3 Planificarea proiectului . . . 4

2 Studiu bibliografic și fundamente teoretice **5**
2.1 Paralelismul la nivel de nucleu . . . 5
2.1.1 Fire de execuție logice (SMT / Hyper-Threading) . . . 5
2.2 Indicatori de performanță utilizați în SSC . . . 5
2.2.1 Speedup (S_p) . . . 5
2.2.2 Eficiența (E_p) . . . 5
2.2.3 Debit de procesare (throughput) . . . 6
2.3 Legea lui Amdahl . . . 6
2.4 Ierarhia memoriei și efectul asupra performanței . . . 6

3 Analiza algoritmilor de benchmark **7**
3.1 Suma pătratelor (SquareSum) . . . 7
3.1.1 Descriere . . . 7
3.1.2 Motivație și impact asupra hardware-ului . . . 7
3.1.3 Pseudocod . . . 7
3.2 Înmulțirea matricelor (MatMul) . . . 8
3.2.1 Descriere . . . 8
3.2.2 Motivație și impact . . . 8
3.3 Estimarea lui π prin Monte Carlo . . . 8
3.3.1 Descriere . . . 8
3.3.2 Motivație și impact . . . 8
3.4 Merge Sort paralelizat . . . 8
3.4.1 Descriere . . . 8
3.4.2 Motivație și impact . . . 8
3.5 Transformata Fourier rapidă (FFT) . . . 8
3.5.1 Descriere . . . 8
3.5.2 Motivație și impact . . . 9

4 Proiectarea sistemului **10**
4.1 Arhitectura generală . . . 10
4.2 Interfața de linie de comandă . . . 10
4.3 Formatul de rezultate JSON Lines (JSONL) . . . 11

5 Implementare tehnică **12**
5.1 Implementare în C . . . 12
5.1.1 Împărțirea sarcinii pe fire (SquareSum) . . . 12
5.1.2 Măsurarea timpului . . . 12
5.1.3 Înmulțirea matricelor (fragment) . . . 13
5.2 Implementare în Rust . . . 13
5.2.1 Paralelizare de nivel înalt (SquareSum) . . . 13
5.2.2 Monte Carlo (fragment) . . . 14
5.3 Dashboard (Python + Streamlit) . . . 15

6 Testare și rezultate experimentale **17**
6.1 Metodologia de testare . . . 17
6.1.1 Configurația de test . . . 17
6.1.2 Parametrii sesiunilor . . . 17
6.2 Exemple de rezultate brute (results.jsonl) . . . 17
6.3 Vizualizarea utilizării nucleelor . . . 18
6.4 Scalabilitate: speedup în funcție de numărul de fire . . . 19
6.5 Interpretarea rezultatelor . . . 19

7 Concluzii și direcții de dezvoltare **21**
7.1 Concluzii . . . 21
7.2 Observații privind C vs Rust . . . 21
7.3 Dezvoltări viitoare . . . 21

Anexe **22**

Resurse bibliografice **23**

Capitolul 1

Introducere

1.1 Contextul actual al arhitecturilor multicore

În etapa actuală a evoluției microprocesoarelor, creșterea performanței nu mai este obținută predominant prin mărirea frecvenței de ceas, ci prin extinderea paralelismului la nivel de nucleu. Această direcție este impusă de limitări fizice: disiparea termică, consumul energetic și eficiența scăzută a scalării frecvenței în tehnologii de fabricație avansate.

Prin urmare, optimizarea aplicațiilor pentru a exploata eficient paralelismul hardware a devenit esențială. Proiectul de față investighează acest aspect prin construirea unui set de programe benchmark capabile să evidențieze limitări tipice (constrângeri de memorie, sincronizare, echilibrare de sarcină), sub mai multe clase de algoritmi.

1.2 Propunerea de proiect

MultiCore Benchmark Studio este un set de programe de testare a performanței unui procesor multicore care implementează algoritmi reprezentativi, aleși astfel încât să solicite diferențiat: unități aritmetico-logice (ALU), unități în virgulă mobilă (FPU) și subsistemul de memorie (cache/RAM).

Obiectivele principale sunt:

- măsurarea timpului de execuție în funcție de numărul de fire de execuție;
- calculul indicatorilor de scalabilitate: **speedup** și **eficiență**;
- compararea a două abordări de programare de sistem: **C** vs **Rust**;
- integrarea unui panou de control (dashboard) pentru orchestrare, colectare și vizualizare a datelor.

Proiectul compară:

- **C**: control explicit al resurselor și utilizarea mecanismelor clasice de threading (de exemplu, POSIX Threads pe Linux; pe Windows se poate folosi API-ul nativ sau un strat de compatibilitate).
- **Rust**: siguranță la nivel de memorie impusă la compilare și paralelizare de nivel înalt prin biblioteci dedicate (de exemplu, `rayon`).

1.3 Planificarea proiectului

Proiectul a fost segmentat în etape pentru a obține stabilitate incrementală:

Etapa	**Săptămâni**	**Obiective**
I	S3–S5	Alegerea algoritmilor și corelarea lor cu componentele hardware (ALU/FPU/memorie). Implementarea inițială în C.
II	S6–S8	Implementarea în Rust și standardizarea formatului de rezultate (JSON Lines) pentru interoperabilitate.
III	S9–S11	Dezvoltarea dashboard-ului (Python + Streamlit): orchestrare teste, telemetrie CPU, vizualizări și tabele.
IV	S12	Rulări experimentale, agregare rezultate, interpretare pe baza indicatorilor de scalabilitate.

Tabela 1.1: Planificarea etapelor proiectului

Capitolul 2

Studiu bibliografic și fundamente teoretice

2.1 Paralelismul la nivel de nucleu

Un procesor modern este alcătuit din mai multe nuclee, fiecare având unități de execuție dedicate (ALU/FPU) și o ierarhie de cache. De regulă, cache-ul L1 și L2 sunt private per nucleu, iar L3 este partajat.

2.1.1 Fire de execuție logice (SMT / Hyper-Threading)

Unele arhitecturi implementează simultan execuția a două fire software pe același nucleu fizic (SMT). Sistemul de operare vede astfel *procesoare logice* suplimentare. Beneficiul apare în special atunci când un fir stagnează (de exemplu, așteaptă date din memorie), permițând celuilalt fir să utilizeze unitățile de execuție. Totuși, îmbunătățirea nu este ideală, deoarece resursele fizice sunt partajate, astfel fiind necesare mecanisme de sincronizare pentru eficientizare.

2.2 Indicatori de performanță utilizați în SSC

2.2.1 Speedup (S_p)

$$S_p = \frac{T_1}{T_p} \tag{2.1}$$

unde T_1 este timpul de execuție pe un singur fir, iar T_p timpul pe p fire.

2.2.2 Eficiența (E_p)

$$E_p = \frac{S_p}{p} \times 100\% \tag{2.2}$$

Eficiența exprimă cât de bine este utilizat paralelismul. Valori mici indică probleme precum sincronizare excesivă, competiție pe memorie sau distribuție neuniformă a sarcinii.

2.2.3 Debit de procesare (throughput)

$$\text{Debit} = \frac{\text{Număr operații}}{T_p} \tag{2.3}$$

Debitul este relevant mai ales pentru algoritmi cu muncă uniformă per element, unde numărul de operații este estimabil.

2.3 Legea lui Amdahl

Legea lui Amdahl oferă o limită teoretică pentru speedup:

$$S_p = \frac{1}{(1-f) + \frac{f}{p}} \tag{2.4}$$

unde f este fracțiunea paralelizabilă. În practică, speedup-ul este afectat suplimentar de:

- costurile de sincronizare;
- latența memoriei ("peretele memoriei");
- invalidări de cache și partajare nefavorabilă (de exemplu, *false sharing*);
- distribuție dezechilibrată a sarcinii.

2.4 Ierarhia memoriei și efectul asupra performanței

Nivel	**Latență (ordine de mărime)**	**Dimensiune tipică**
Registre CPU	sub 1 ciclu	zeci de registre
Cache L1	3–5 cicluri	32–64 KB / nucleu
Cache L2	10–20 cicluri	sute de KB / nucleu
Cache L3	zeci de cicluri	MB partajat
RAM	sute de cicluri	GB

Tabela 2.1: Ierarhia de memorie într-un sistem modern (valori orientative)

Capitolul 3

Analiza algoritmilor de benchmark

Alegerea algoritmilor urmărește acoperirea unor tipare diferite de încărcare: **calc. aritmetic simplu** (ALU), **calc. numeric** (FPU), **acces intens la memorie** (cache/RAM) și **control logic** (predicția ramificațiilor).

3.1 Suma pătratelor (SquareSum)

3.1.1 Descriere

Se calculează:

$$\sum_{i=0}^{N-1} i^2 \tag{3.1}$$

3.1.2 Motivație și impact asupra hardware-ului

Algoritmul este independent pe iterații (paralelizare directă). Solicită preponderent ALU, cu acces redus la memorie și sincronizare minimă (doar la agregarea sumelor parțiale). Este util pentru a observa costul "pur" al gestionării firelor de execuție.

3.1.3 Pseudocod

```
function parallelSquareSum(N, p):
    chunk = N / p
    partial[p] = {0}

    parallel for tid in 0..p-1:
        start = tid * chunk
        if (tid == p-1)
        end = N
        else end = (tid+1)*chunk
        for i in start..end-1:
            partial[tid] += i*i

    return sum(partial)
```

Listing 3.1: Pseudocod pentru SquareSum

3.2 Înmulțirea matricelor (MatMul)

3.2.1 Descriere

$$C[i][j] = \sum_{k=0}^{n-1} A[i][k] \times B[k][j] \tag{3.2}$$

3.2.2 Motivație și impact

Este un caz clasic de încărcare mixtă: calcule în virgulă mobilă și acces intens la memorie. Poate evidenția saturarea lățimii de bandă a memoriei și efectul ierarhiei de cache, în funcție de dimensiunea matricelor și de strategiile de acces (de exemplu, blocare pe cache).

3.3 Estimarea lui π prin Monte Carlo

3.3.1 Descriere

Se generează puncte aleatoare (x, y) în $[0, 1]^2$ și se numără cele care satisfac $x^2 + y^2 \leq 1$.

$$\pi \approx 4 \cdot \frac{\text{puncte în cerc}}{N} \tag{3.3}$$

3.3.2 Motivație și impact

Sarcină predominant numerică (FPU), cu acces redus la memorie. Paralelizarea este eficientă deoarece experimentele sunt independente. Performanța poate fi influențată și de generatorul de numere pseudo-aleatoare și de strategia de inițializare per fir.

3.4 Merge Sort paralelizat

3.4.1 Descriere

Sortare de tip divide-et-impera, cu complexitate $O(n \log n)$.

3.4.2 Motivație și impact

Este un algoritm reprezentativ pentru sarcini reale, caracterizat de accesuri neregulate la memorie și multe comparații. Astfel, poate evidenția:

- latența cache-ului și rata de ratări în cache;
- limitări ale predicției ramificațiilor;
- costuri de sincronizare/îmbinare între subsecțiuni.

3.5 Transformata Fourier rapidă (FFT)

3.5.1 Descriere

Implementarea tipică (Cooley–Tukey) reduce complexitatea la $O(N \log N)$ față de $O(N^2)$.

3.5.2 Motivație și impact

Modelul de acces ("butterfly") poate solicita transferuri importante între nivele de cache și introduce etape de sincronizare între fire, fiind relevant pentru observarea costurilor de coordonare și a lățimii de bandă efective.

Capitolul 4

Proiectarea sistemului

4.1 Arhitectura generală

Sistemul este organizat în trei componente:

1. **Dashboard (Python + Streamlit)**: configurează testele, lansează execuții, colectează și prezintă date.
2. **Motoare native de calcul (C și Rust)**: execută algoritmii și raportează rezultatele.
3. **Persistența rezultatelor (results.jsonl)**: stocare incrementală, robustă la întreruperi.

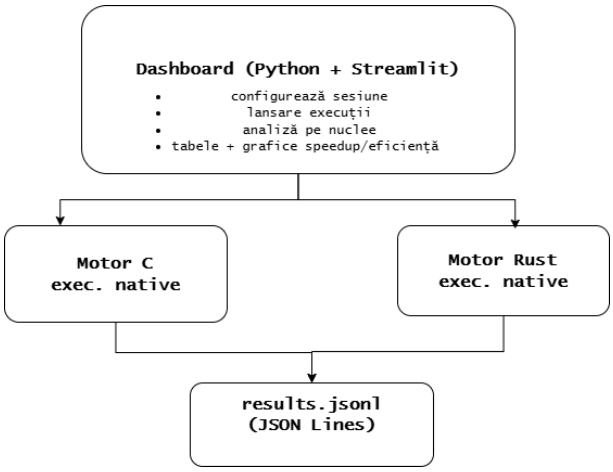

Figura 4.1: Arhitectura generală a sistemului

4.2 Interfața de linie de comandă

Motoarele sunt apelate cu parametri standardizați (algoritm, dimensiune, număr de fire), iar rezultatul este produs într-un format ușor de prelucrat automat (JSON).

```
.\target\release\ssc_benchmark_rs.exe --algo 5 --threads 8 --runs 5 --
    out results_rs.jsonl
```

Listing 4.1: Exemplu apel din linia de comandă

4.3 Formatul de rezultate JSON Lines (JSONL)

Fiecare execuție produce o singură linie JSON. Avantaje:

- scriere incrementală (append-only);
- risc redus de corupere a datelor la întreruperi;
- procesare facilă în Python (citire linie cu linie / agregare în DataFrame).

```
{
  "language": "c",
  "alg": 1,
  "threads": 4,
  "run_index": 2,
  "input_size": 1000000,
  "seconds": 0.000604
}
```

Listing 4.2: Structură tipică a unei înregistrări JSONL

Capitolul 5

Implementare tehnică

5.1 Implementare în C

5.1.1 Împărțirea sarcinii pe fire (SquareSum)

În implementarea C, fiecare fir primește un interval disjunct și produce un rezultat parțial, agregat ulterior de firul principal.

```c
#define _CRT_SECURE_NO_WARNINGS
#include "utils.h"
#include <windows.h>
#include <stdint.h>
#include <stdlib.h>

typedef struct SumSqTask{
    uint64_t start;
    uint64_t end;
    double result;
} SumSqTask;

static DWORD WINAPI sumsq_worker(LPVOID arg) {
    SumSqTask* t = (SumSqTask*)arg;
    double s = 0.0;

    for (uint64_t i = t->start; i < t->end; i++) {
        double x = (double)i;
        s += x * x;
    }

    t->result = s;
    return 0;
}
```

Listing 5.1: Structuri și funcție de lucru pentru SquareSum (C)

5.1.2 Măsurarea timpului

Pentru măsurători, se utilizează un ceas monoton (în funcție de platformă). Mai jos este un exemplu POSIX (pe Linux). Pe Windows se poate utiliza un echivalent (de exemplu, performanță înaltă).

```c
#include <time.h>
```

```c

double now_ms(void) {
    struct timespec ts;
    clock_gettime(CLOCK_MONOTONIC, &ts);
    return (double)ts.tv_sec * 1000.0 + (double)ts.tv_nsec / 1e6;
}
```

Listing 5.2: Măsurarea timpului (exemplu POSIX)

5.1.3 Înmulțirea matricelor (fragment)

```c
typedef struct MatMulTask{
    int tid;
    int threads;
    int n;
    const double* A;
    const double* B;
    double* C;
    int row_start;
    int row_end;
} MatMulTask;

static DWORD WINAPI matmul_worker(LPVOID arg) {
    MatMulTask* t = (MatMulTask*)arg;
    int n = t->n;
    const double* A = t->A;
    const double* B = t->B;
    double* C = t->C;

    for (int i = t->row_start; i < t->row_end; i++) {
        int iN = i * n;
        for (int j = 0; j < n; j++) {
            double sum = 0.0;
            for (int k = 0; k < n; k++) {
                sum += A[iN + k] * B[k * n + j];

            }
            C[iN + j] = sum;
        }

    }
    return 0;
}
```

Listing 5.3: Fragment MatMul paralelizat pe intervale de rânduri (C)

5.2 Implementare în Rust

5.2.1 Paralelizare de nivel înalt (SquareSum)

În Rust, paralelizarea este realizată printr-un model de tip “work stealing”, care redistribuie sarcinile pentru o încărcare mai uniformă a nucleelor.

```rust
fn run_sumsq(size: u64, threads: usize) {
    use rayon::prelude::*;
```

```rust

    let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(threads)
        .build()
        .expect("failed to build rayon thread pool");

    let sum: f64 = pool.install(|| {
        (0u64..size)
            .into_par_iter()
            .map(|i| {
                let x = i as f64;
                x * x
            })
            .sum()
    });

    std::hint::black_box(sum);
}
```

Listing 5.4: SquareSum în Rust cu Rayon

5.2.2 Monte Carlo (fragment)

```rust
fn run_montecarlo(size: u64, threads: usize) {
    use rayon::prelude::*;
    use rand::rngs::SmallRng;
    use rand::{Rng, SeedableRng};

    let pool = rayon::ThreadPoolBuilder::new()
        .num_threads(threads)
        .build()
        .expect("failed to build rayon thread pool");

    let inside: u64 = pool.install(|| {

        let chunk = (size / threads as u64).max(1);
        (0..threads)
            .into_par_iter()
            .map(|tid| {
                let start = tid as u64 * chunk;
                let end = if tid == threads - 1 {
                    size
                } else {
                    ((tid as u64 + 1) * chunk).min(size)
                };

                let mut rng = SmallRng(tid as u64 + 1));

                let mut local_inside = 0u64;
                for _ in start..end {
                    let x: f64 = rng.r#gen::<f64>();
                    let y: f64 = rng.r#gen::<f64>();
                    if x * x + y * y <= 1.0 {
                        local_inside += 1;
                    }
                }
                local_inside
```

```
            })
            .sum()
    });

    let pi = 4.0 * (inside as f64) / (size as f64);
    std::hint::black_box(pi);
}
```

Listing 5.5: Estimarea lui pi (Monte Carlo) în Rust

5.3 Dashboard (Python + Streamlit)

Dashboard-ul este responsabil de configurarea sesiunii, lansarea execuțiilor, colectarea rezultatelor și prezentarea lor sub formă de grafice și tabele.

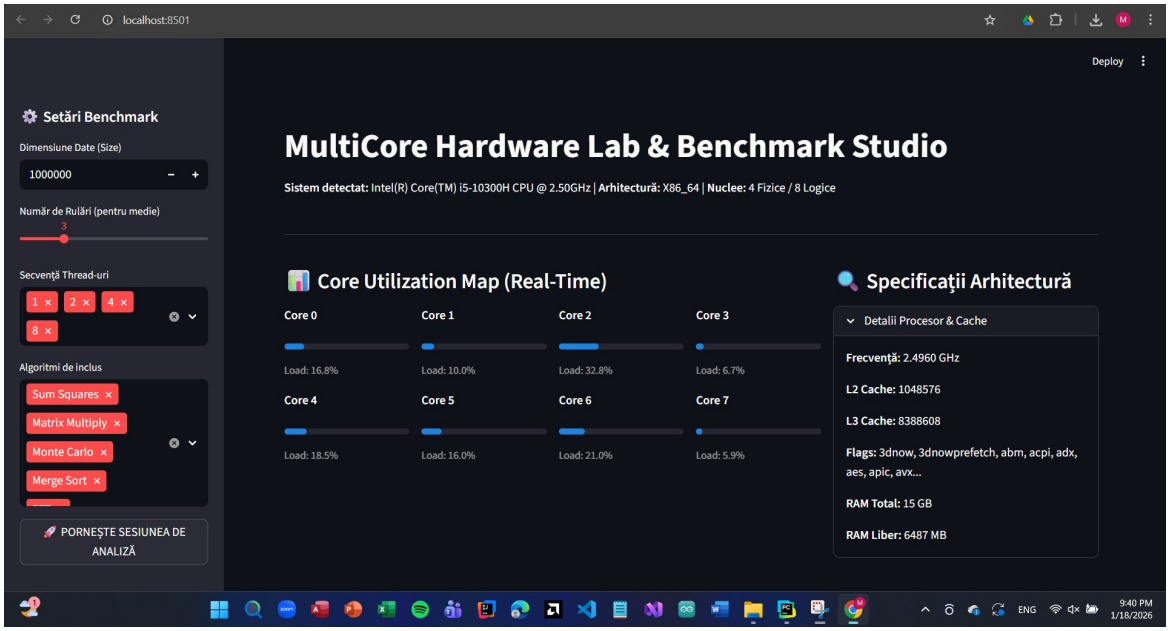

Figura 5.1: Interfața dashboard: configurarea benchmark-ului și monitorizare în timp real pe nuclee

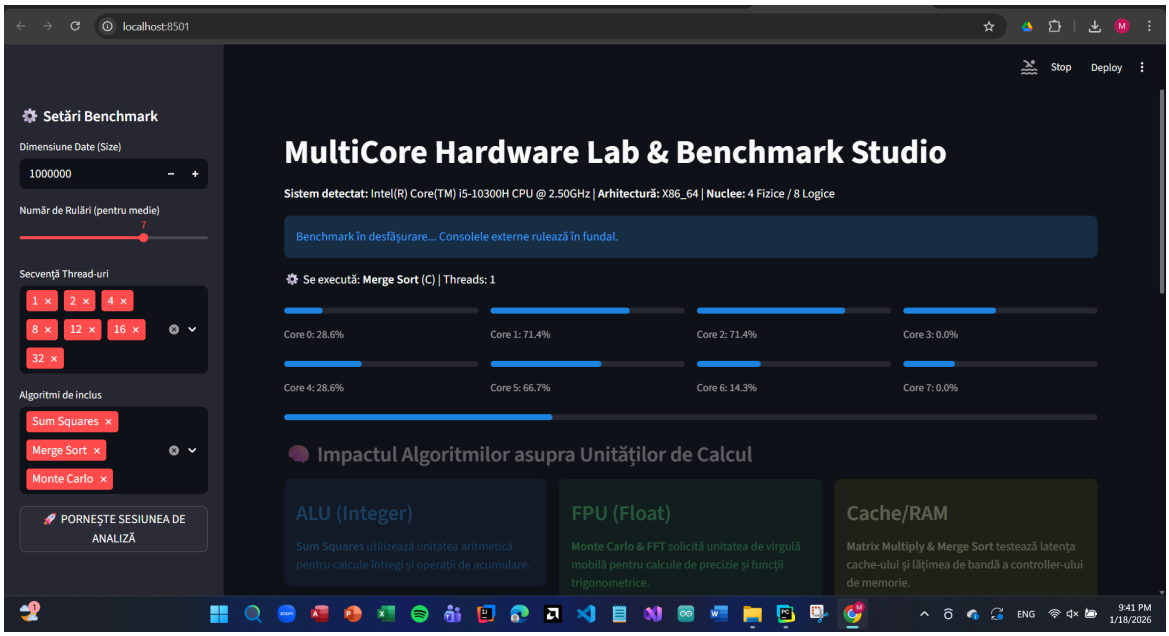

Figura 5.2: Sesiune de benchmark în execuție: afișarea algoritmului curent și utilizarea nucleelor

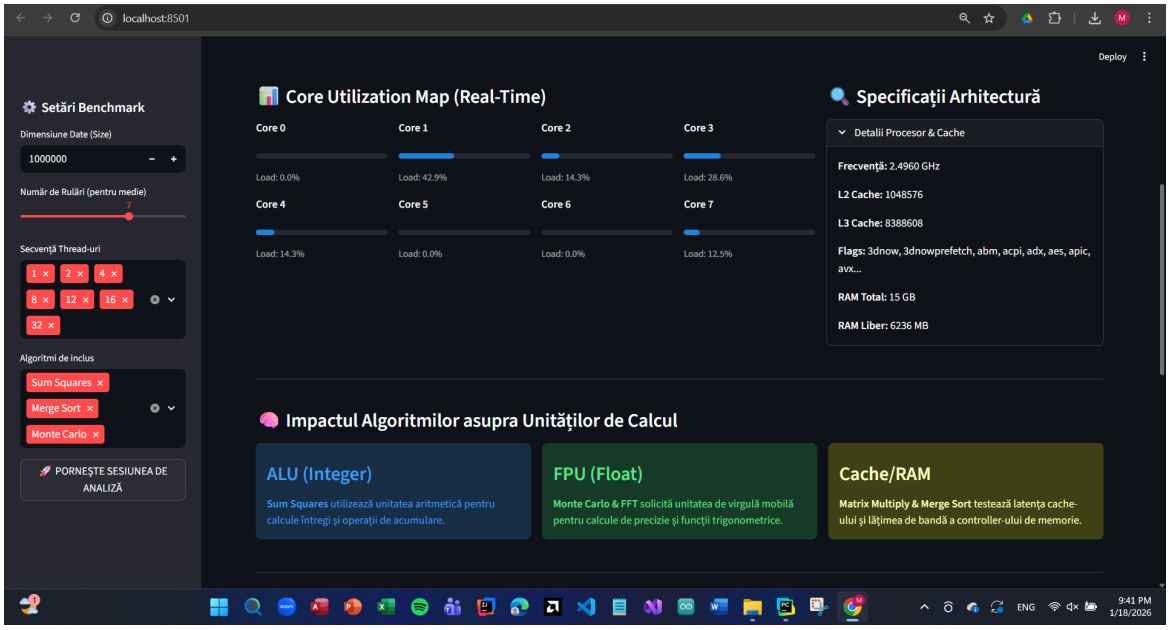

Figura 5.3: Panoul de impact: corelarea algoritmilor cu unitățile de calcul (ALU/FPU/-memorie)

Capitolul 6

Testare și rezultate experimentale

6.1 Metodologia de testare

6.1.1 Configurația de test

În cadrul rulărilor prezentate, sistemul detectat de dashboard este:

Componentă	**Specificații**
Procesor	Intel Core i5-10300H @ 2.50 GHz
Arhitectură	x86-64
Nuclee fizice	4
Fire logice	8
Cache L2 (raportat)	$\approx 1\,048\,576B$
Cache L3 (raportat)	$\approx 8\,388\,608B$
Memorie RAM totală	15 GB
Sistem de operare	Windows 11

Tabela 6.1: Configurația utilizată pentru rulările ilustrate în dashboard

6.1.2 Parametrii sesiunilor

Testele se rulează cu:

- dimensiunea datelor (ex.: 10^6 elemente);
- număr de rulări pentru medie;
- o secvență de valori pentru numărul de fire (de exemplu 1, 2, 4, 8, 12, 16, 32 – în funcție de configurația setată).

6.2 Exemple de rezultate brute (results.jsonl)

Fișierul `results.jsonl` conține câte o linie JSON per execuție.

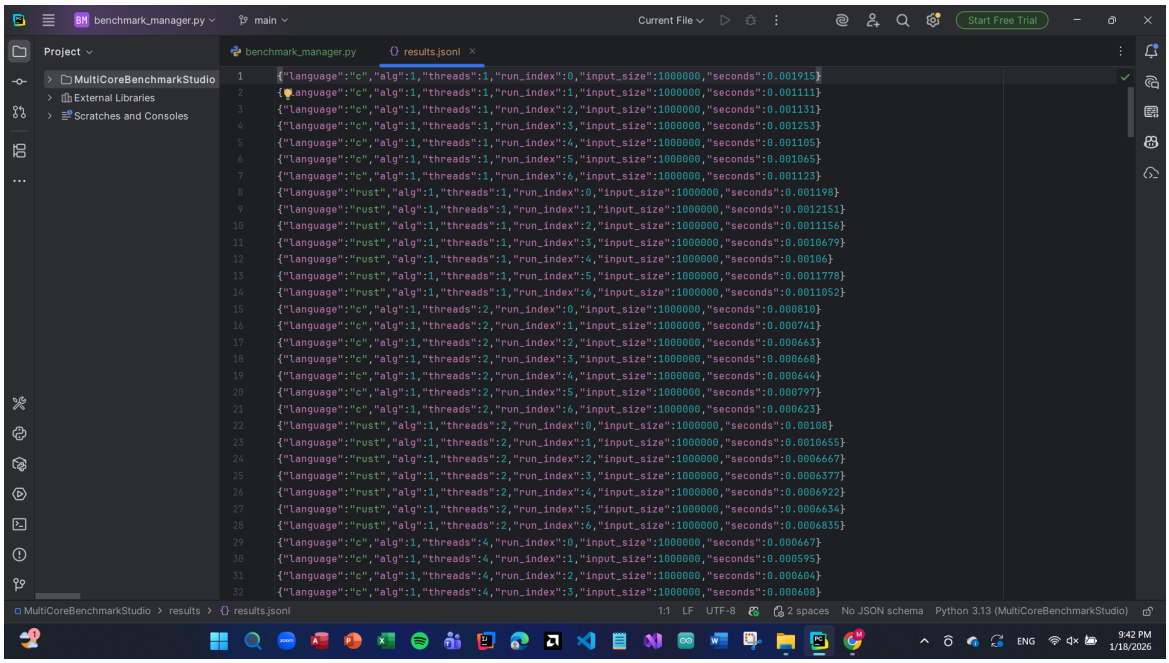

Figura 6.1: Exemplu de înregistrări brute în fișierul results.jsonl

6.3 Vizualizarea utilizării nucleelor

Harta de utilizare pe nuclee permite verificarea încărcării pe firele logice și identificarea situațiilor în care distribuția sarcinii este neuniformă.

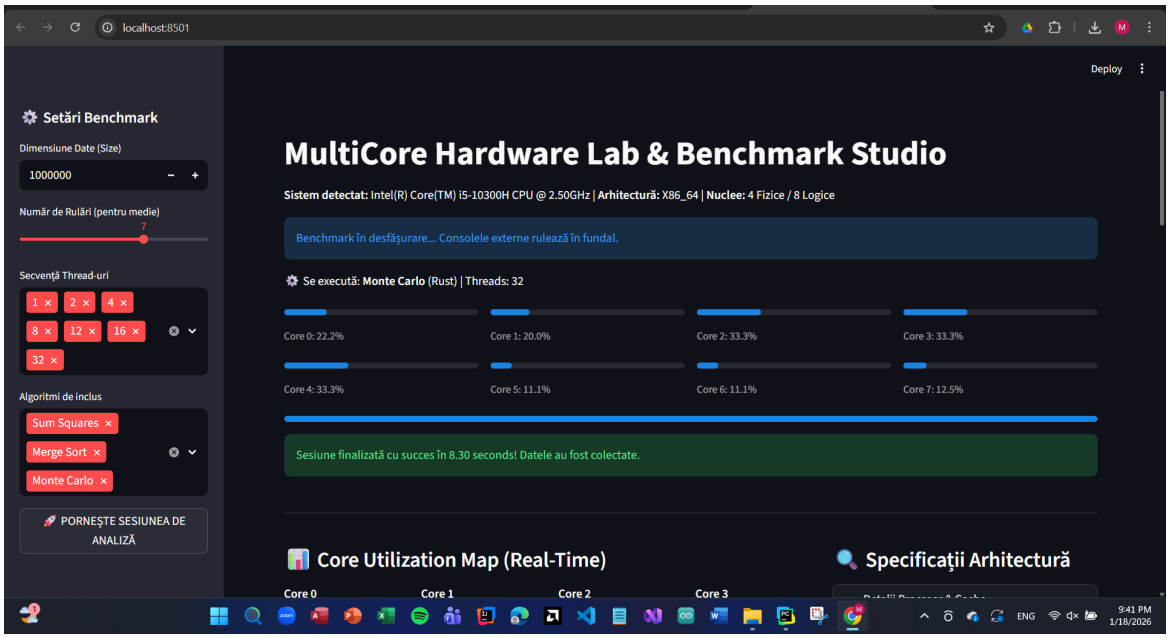

Figura 6.2: Utilizarea nucleelor în timpul unei execuții (exemplu)

6.4 Scalabilitate: speedup în funcție de numărul de fire

Graficul de speedup compară implementarea C cu implementarea Rust și include linia de scalabilitate ideală ($S_p = p$).

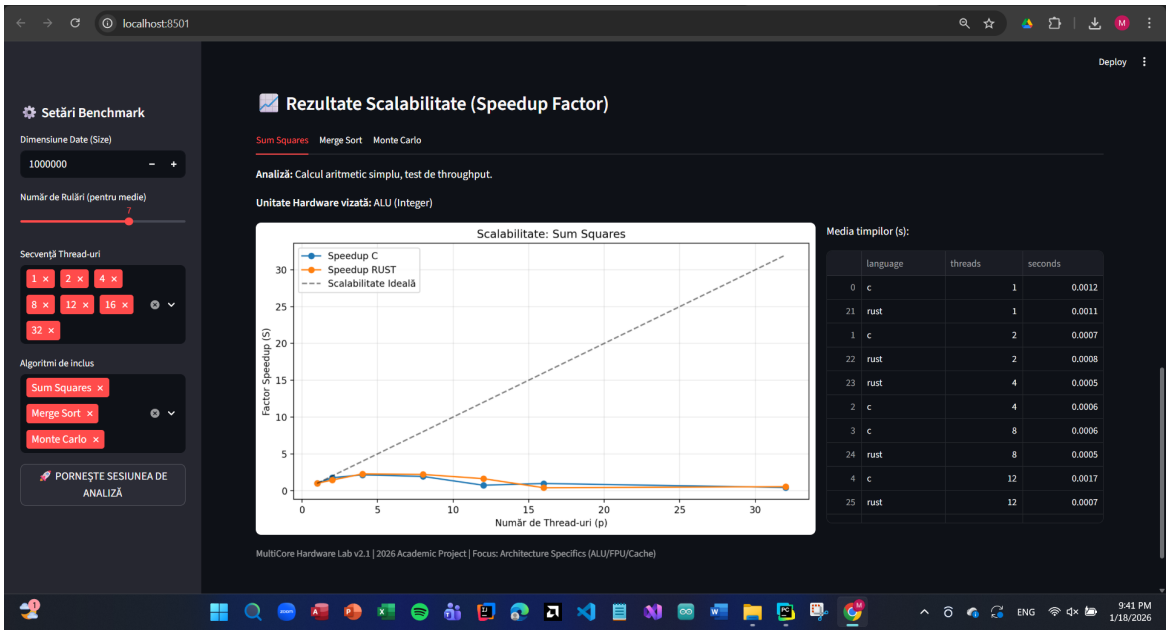

Figura 6.3: Rezultate de scalabilitate (exemplu): speedup pentru “Suma pătratelo”

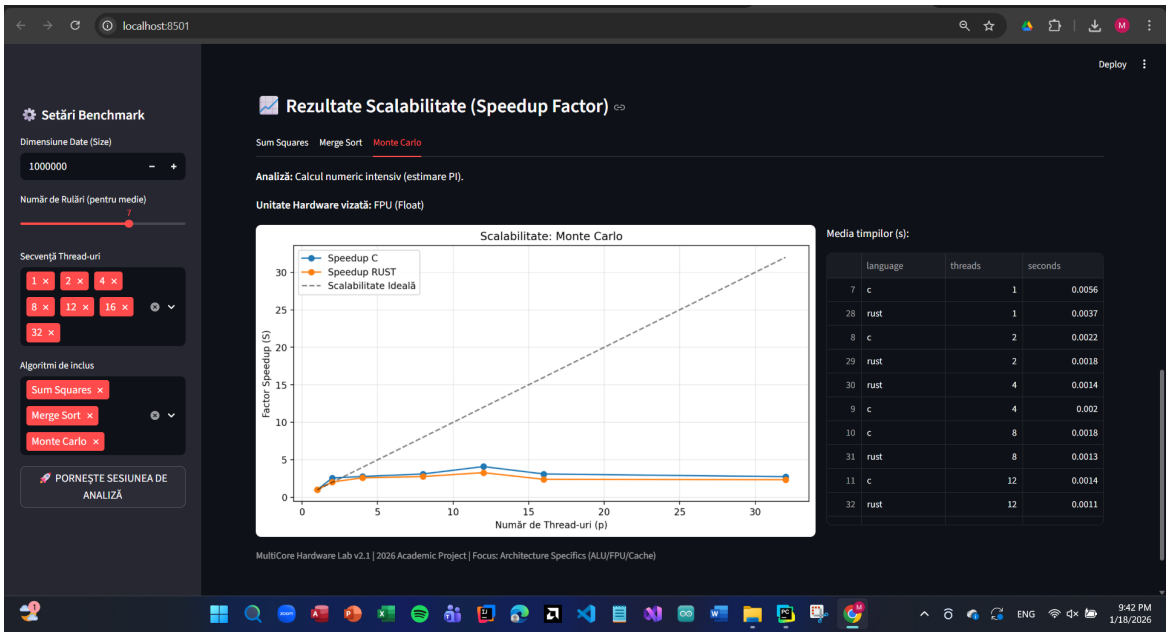

Figura 6.4: Rezultate de scalabilitate (exemplu): speedup pentru “Monte Carlo”

6.5 Interpretarea rezultatelor

În general, algoritmii cu iterații independente (de tip SquareSum, Monte Carlo) tind să aibă scalabilitate mai bună decât algoritmii dominați de memorie sau control logic (de

exemplu, MatMul fără optimizări de localitate sau Merge Sort cu accesuri neregulate).

Pe platforma ilustrată (4 nuclee fizice / 8 fire logice), este de așteptat ca îmbunătățirea să se apropie de ideal până la ocuparea nucleelor fizice, iar după aceea beneficiul suplimentar oferit de firele logice să fie mai redus.

Capitolul 7

Concluzii și direcții de dezvoltare

7.1 Concluzii

Proiectul MultiCore Benchmark Studio confirmă, prin măsurători experimentale, faptul că scalabilitatea depinde simultan de:

- fracțiunea paralelizabilă din cod (Amdahl);
- costurile de gestionare a firelor și de sincronizare;
- comportamentul ierarhiei de memorie (cache/RAM) și competiția între nuclee;
- echilibrul de sarcină între fire.

7.2 Observații privind C vs Rust

Compararea C și Rust evidențiază două perspective:

- **C** oferă control explicit și este potrivit pentru optimizări foarte fine, dar necesită grijă sporită la corectitudine (memorie, concurență).
- **Rust** impune reguli stricte de siguranță la compilare și poate reduce riscurile de erori concurente; în plus, paralelizarea de nivel înalt poate simplifica implementarea.

7.3 Dezvoltări viitoare

Posibile extinderi:

1. optimizări de localitate (blocare pe cache) pentru MatMul;
2. utilizarea instrucțiunilor vectoriale (SIMD) pentru sarcini numerice;
3. controlul afinității firelor pentru analize mai fine asupra planificării;
4. extinderea setului cu benchmark-uri suplimentare (I/O, memorie, latență).

Anexe

Anexa A: Instrucțiuni de rulare

A.1 Compilare

C (exemplu Linux):

```
gcc -O3 -pthread -march=native -o benchmark_c main.c -lm
```

Rust:

```
cargo build --release
```

A.2 Dashboard

```
pip install streamlit pandas matplotlib psutil
streamlit run dashboard.py
```

Anexa B: Structura proiectului

```
MultiCore-Benchmark-Studio/
 bench_c/            (surse C)
 bench_rust/         (surse Rust)
 dashboard/              (Python + Streamlit)
 results.jsonl
 images/
     Logo_UTCN.png
     Dashboard1.jpg
     Dashboard2.png
     ...
     Dashboard8.png
```

Resurse bibliografice

1. Amdahl, G. M. (1967). *Validity of the single processor approach to achieving large scale computing capabilities*. AFIPS.
2. Gustafson, J. L. (1988). *Reevaluating Amdahl's Law*. Communications of the ACM.
3. Hennessy, J. L., Patterson, D. A. (2017). *Computer Architecture: A Quantitative Approach*. Morgan Kaufmann.
4. Williams, S. et al. (2009). *Roofline: An insightful visual performance model for multicore architectures*. Communications of the ACM.
5. Documentația `rayon`: https://docs.rs/rayon/
6. SPEC CPU Benchmark Suite: https://www.spec.org/
7. Materiale de curs/laborator: *Structura Sistemelor de Calcul*, UTCN.